Table of contents

Table of contents 1
Team info 1
Project overview 2
Scope description 3
Selected pentesting methodology 5
Scoring system – CVSS 4.0 6
Threat model 8
Intelligence-gathering outcomes 9
Public Availability 9
Authentication 11
Identified Technologies (via reconnaissance) 13
Assets Identified 18
External Integrations 19
Hosting & Infrastructure Observations 21
Entry Points 23
High-Level Attack Surface 25
Attack Surface Indicators 26
Conclusion 26
Structure of the final report 27

Team info

1. Name: Hurtovyi Danylo
   Nickname: hurtodan
   Email: hurtodan@fit.cvut.cz
2. Name: Pysariev Rostyslav
   Nickname: pysarros
   Email: pysarros@fit.cvut.cz

Project overview

Item	Detail
Purpose	The purpose of this project is to perform a penetration test of the CS.MONEY platform. The goal is to identify potential security vulnerabilities and evaluate the overall security posture of the system.
Audience & Access	• Public URL: https://cs.money • The platform is publicly accessible. • Certain functionalities require authentication via a Steam account.
Tech at a Glance	• Frontend: Next.js (React), Bootstrap, Animate.css • Backend: REST API (likely Node.js – Express/NestJS) • Database: MySQL • Infrastructure: AWS (Amazon Web Services), Cloudflare (CDN, DDoS protection) • Analytics: Google Analytics, Amplitude, Hotjar, OpenTelemetry • Payments: PayPal, BitPay, BridgerPay • Security: HSTS, Cloudflare Turnstile • Other: WordPress (blog), Zendesk (support system)
Key Features	• Trading of in-game items (CS skins) • User accounts and authentication • Inventory management • Pricing and trading system
Data & Hosting	The platform is hosted on AWS cloud infrastructure and uses Cloudflare as a CDN and security layer. User data, transactions, inventory, and pricing information are processed through a REST API and stored in a MySQL database. Additional services include analytics tracking (Google Analytics, Amplitude), customer support (Zendesk), and payment processing systems (PayPal, BitPay).

This penetration test is conducted as part of the BI-EHA course at CTU FIT. The tested system is part of a public bug bounty program on the HackerOne platform. All testing activities will be performed in accordance with the program rules and will be limited to allowed targets and techniques. No denial-of-service or other prohibited attacks will be performed.

Scope description

This engagement is a **black-box** security assessment of the CS.MONEY platform, conducted through its public bug bounty program on HackerOne. Testers have access only to publicly available interfaces and functionalities, with no access to source code or internal infrastructure.

In-scope

- Primary application:
  - https://cs.money
- Additional domains:
  - https://support.cs.money (support web client)
  - https://wiki.cs.money (information platform)
  - https://3d.cs.money (3D skin viewer)
- Blog:
  - https://blog.cs.money
    - Only vulnerabilities in third-party or built-in WordPress plugins are considered in scope
    - WordPress core, themes, and configuration issues are out of scope
- Allowed testing activities:
  - Dynamic application testing (frontend, API endpoints, WebSocket communication)
  - Authentication and session management testing (Steam login, tokens, cookies)
  - Authorization and access control validation
  - Input validation testing (XSS, IDOR, injection attacks)
  - Business logic testing (trading system, balance handling, inventory management)

Out-of-Scope

- Infrastructure-level attacks:
  - Servers, network devices, Docker/Kubernetes, TLS configuration
- Denial-of-Service (DoS) or resource exhaustion attacks
- Social engineering or phishing attacks
- Brute-force or rate-limit abuse
- Testing accounts not owned by the tester

- The following domains:
  - old.cs.money
  - grafana.cs.money
  - CS.Money Antiscam browser extension
- WordPress core vulnerabilities, themes, and configuration issues

All testing must comply with the HackerOne program rules and be limited to actions that do not negatively impact service availability or user data.

Selected pentesting methodology

We followed the OWASP Web Security Testing Guide (WSTG) v4.2 as the primary framework for this black-box assessment of the CS.MONEY platform.

All testing activities were mapped to relevant WSTG categories to ensure systematic and structured coverage of the application:

- Information Gathering (WSTG-INFO)
- Client-side Testing (WSTG-CLNT)
- API Testing (WSTG-APIT)
- Input Validation (WSTG-INPV)
- Error Handling (WSTG-ERRH)
- Configuration and Deployment Testing (WSTG-CONF)
- Identity Management Testing (WSTG-IDNT)
- Authentication Testing (WSTG-AUTHN)
- Authorization Testing (WSTG-AUTHZ)
- Session Management Testing (WSTG-SESS)
- Cryptography Testing (WSTG-CRYP)
- Business Logic Testing (WSTG-BUSL)

Given the black-box nature of the engagement, testing was performed exclusively through publicly available interfaces, including the web application, API endpoints, and related subdomains defined in scope. In practice, we combined manual testing techniques with automated tools to identify potential vulnerabilities. Test cases were prioritized based on risk level, exposure, and relevance to the platform's core functionality (trading system, user accounts, and financial operations).

Scoring system – CVSS 4.0

The Common Vulnerability Scoring System (CVSS) version 4.0, released on November 1, 2023, is the industry standard for measuring technical severity of software vulnerabilities. This system produces a numeric score from 0.0 (None) to 10.0 (Critical) and provides a vector string that records each input choice.

The score is calculated using the CVSS calculator

Metric Groups

CVSS 4.0 is built from four metric groups:

1. Base - Intrinsic properties of the vulnerability that never change.
2. Threat - Real-world exploitation status (Exploit Maturity).
3. Environmental - How local business context modifies impact (e.g., data criticality, compensating controls).
4. Supplemental - Extra context such as Safety or Automatable; recorded for clarity but not scored.

Base Metrics

- Exploitability - AV (Attack Vector), AC (Attack Complexity), AT (Attack Requirements, new in v4), PR (Privileges Required), UI (User Interaction).
- Impact - split into two tiers: VC/VI/VA (effects on the vulnerable system) and SC/SI/SA (effects on systems subsequently impacted).

Scoring Flow

1. Base Score (CVSS-B) - calculated solely from Base metrics, assuming a reasonable worst-case environment.
2. Base + Threat (CVSS-BT) - Base score adjusted by Exploit Maturity (E). If E is unknown, it defaults to X and is ignored.
3. Base + Threat + Environmental (CVSS-BTE) - final score after overlaying local requirements (CR/IR/AR) and any modified metrics (MAV, MPR, etc.). Scores are rounded to the nearest 0.1 for reporting; full precision is retained in the vector.

Qualitative Severity Ratings

Range	Rating
0.0	None
0.1 – 3.9	Low
4.0 – 6.9	Medium
7.0 – 8.9	High
9.0 – 10.0	Critical

Vector String Format

Every assessment is stored as a text vector beginning with "CVSS:4.0", followed by Base metrics (obligatory) and optional Threat, Environmental, and Supplemental metrics in the mandated order.

Example:

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N/E:P

Unspecified optional metrics are treated as X (Not Defined) and assume the conservative value in calculations.

Usage in This Engagement

All findings for this engagement will be scored with CVSS 4.0. We will publish both the numeric CVSS-BTE value and the complete vector string so that stakeholders (teachers) can reproduce or tailor the score to their own environment without recalculating from scratch.

Threat model

This section identifies what we need to protect on the cs.money platform, who might try to break it, and how they could do so. It will guide all subsequent testing steps.

Assets (what we protect):

- User identities
- Transaction and trading data (CS2 skins trading information)
- Task and trading data (intellectual property and personal user data)
- Session tokens (SAML assertions, JWT cookies)

Primary Actors (who could attack):

1. Unauthorized Internet User – Has network-level access to HTTPS endpoints only.
2. Authenticated User – Normal privileges but could try to alter or modify other users' data.
3. Malicious Admin – Elevated role, could craft fake transactions, modify task data, and execute unauthorized code.
4. Compromised Third-Party Service – Could lead to attacks via malicious SSO IdP or external JS/CSS CDN delivering tampered payloads.

Trust Boundaries:

- Public Internet – Nginx/SPA
- SPA – ReactJS frontend (REST/JSON endpoints, WebSocket)
- API – Backend API (MySQL and Spring Boot)
- Browser – In-browser WebAssembly runtime

High-Level Attack Surfaces:

- Public REST/JSON endpoints – Attack vectors like parameter tampering and IDOR.
- SAML SSO login – Attacks on authentication (e.g., replay attacks, signature stripping).
- WebAssembly – Exploiting the WebAssembly runtime (WASM module poisoning, sandbox escape).
- File Upload Endpoints – Exfiltrating or modifying task assets and attachments.

- Client-Side Storage – Attacks on tokens stored in LocalStorage/SessionStorage (session hijacking).

Threat Categories (STRIDE-style):

Spoofing – Forged SSO assertions.

Tampering – Modifying submissions, trading data, or user task information.

Repudiation – Injecting or hiding logs to cover up unauthorized actions.

Information Disclosure – Accessing private user data or private transaction information.

Denial of Service – Attacking the API or services (e.g., SQL injection leading to downtime or service disruption).

Escalation of Privilege – Unauthorized user escalating privileges (e.g., normal user – admin).

Key Assumptions & Limits:

- This is a black-box test with no access to the source code, database, or OS/host systems.
- Infrastructure hardening (e.g., firewall, DB encryption) is handled by cs.money IT team and will only be referenced if findings in code affect these areas (e.g., hard-coded DB credentials or API keys).

Intelligence-gathering outcomes

Public Availability

The cs.money platform is publicly accessible via the Internet and is hosted at https://cs.money. The application is available over HTTPS (port 443), ensuring encrypted communication between the client and the server.

The secure connection and valid TLS certificate issued by Let's Encrypt were verified through browser inspection (see Figure 1 and Figure 2).

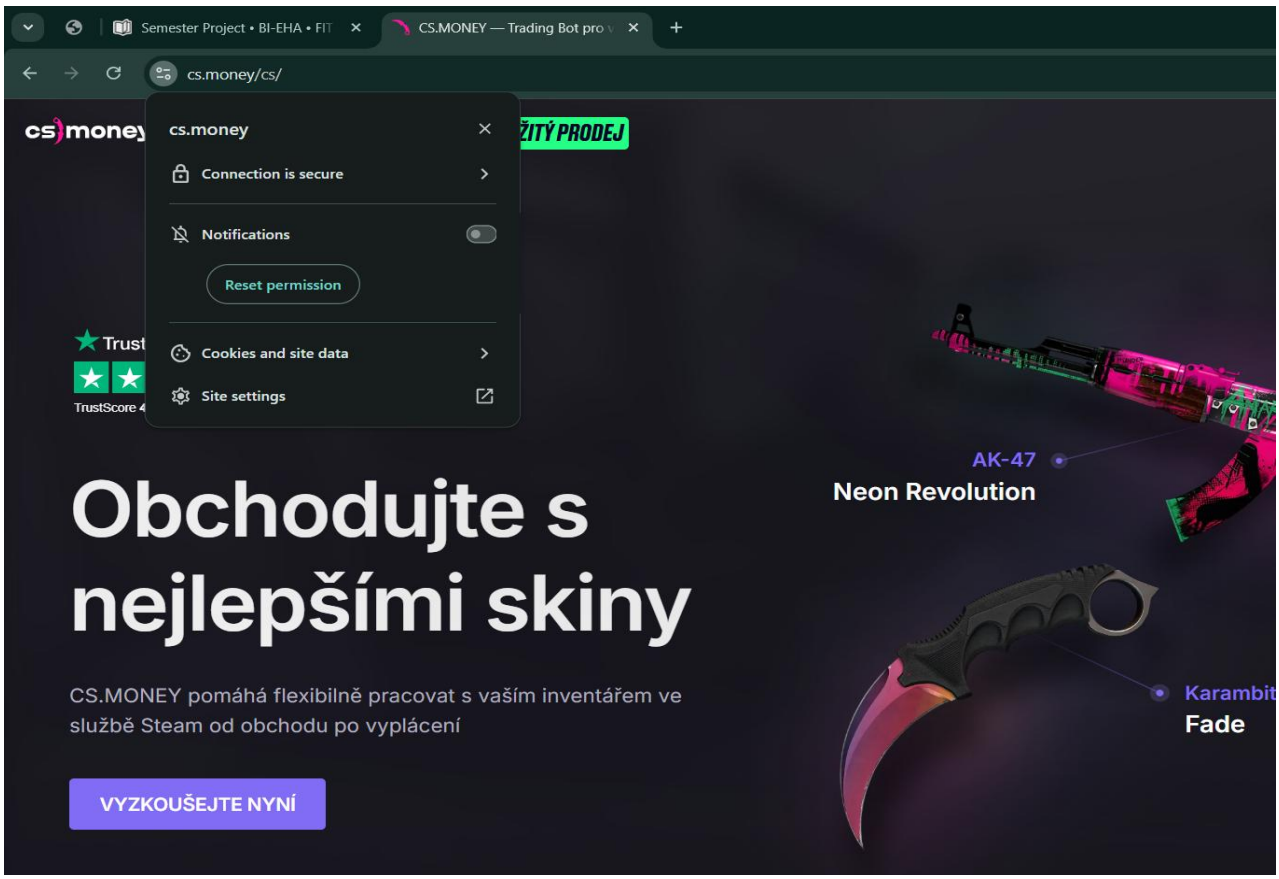

Figure 1: Public availability of cs.money over HTTPS

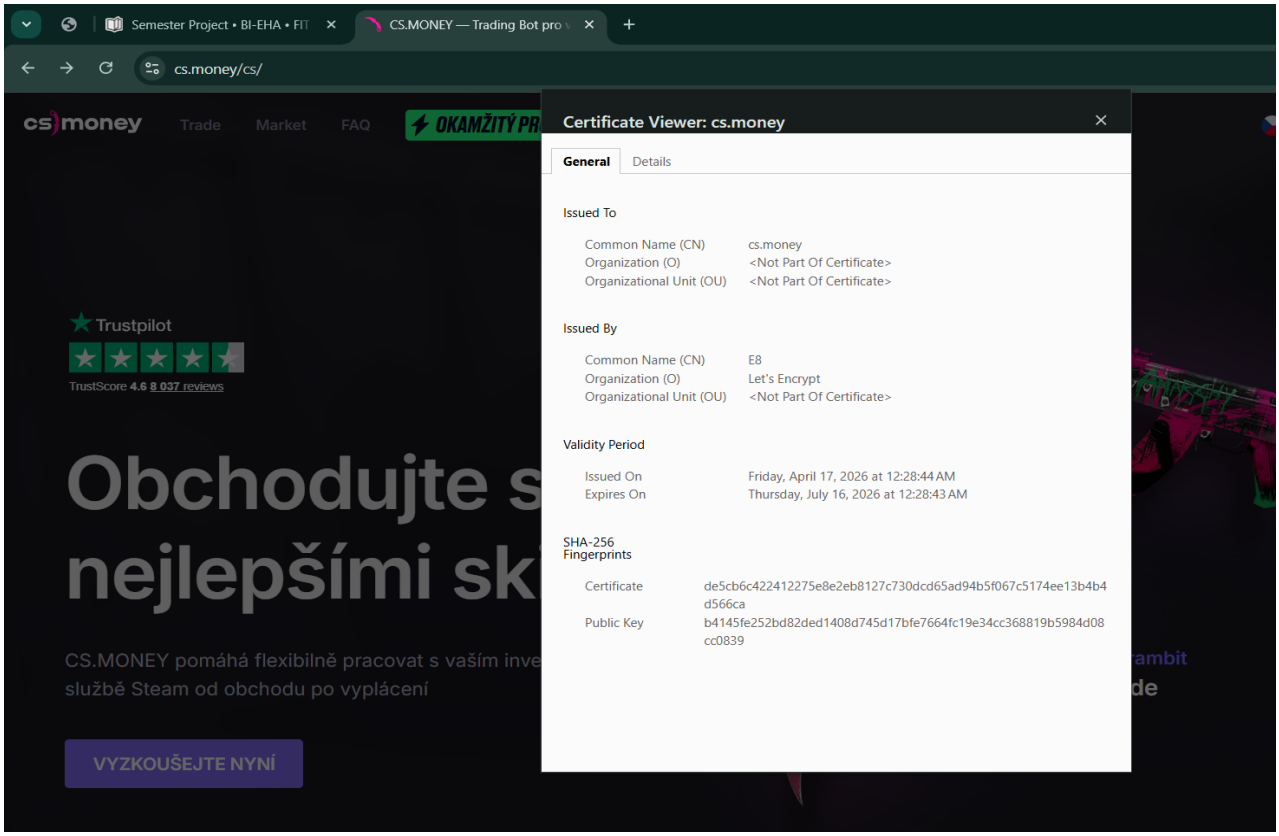

Figure 2: TLS certificate details issued by Let's Encrypt

Authentication

The cs.money platform uses third-party authentication via Steam, based on the OpenID protocol.

During the login process, users are first redirected to an intermediate authentication service (auth.dota.trade), which handles the authentication flow. This service then redirects users to the official Steam OpenID endpoint.

After successful authentication, the user is redirected back to the cs.money platform, where a session is established.

The authentication process was analyzed using Burp Suite, confirming the use of a redirect-based authentication mechanism and external identity provider (see Figures 3-4).

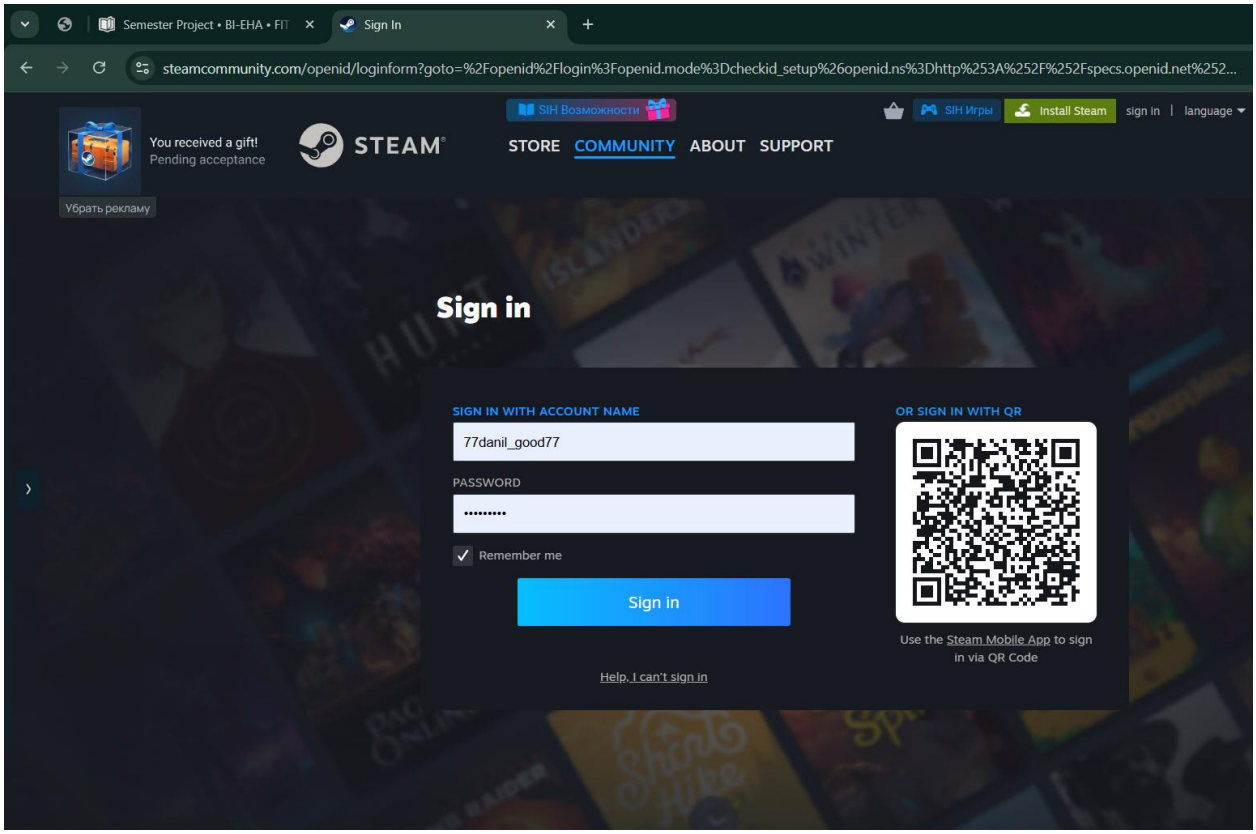

Figure 3: Redirection to Steam OpenID login page during authentication process

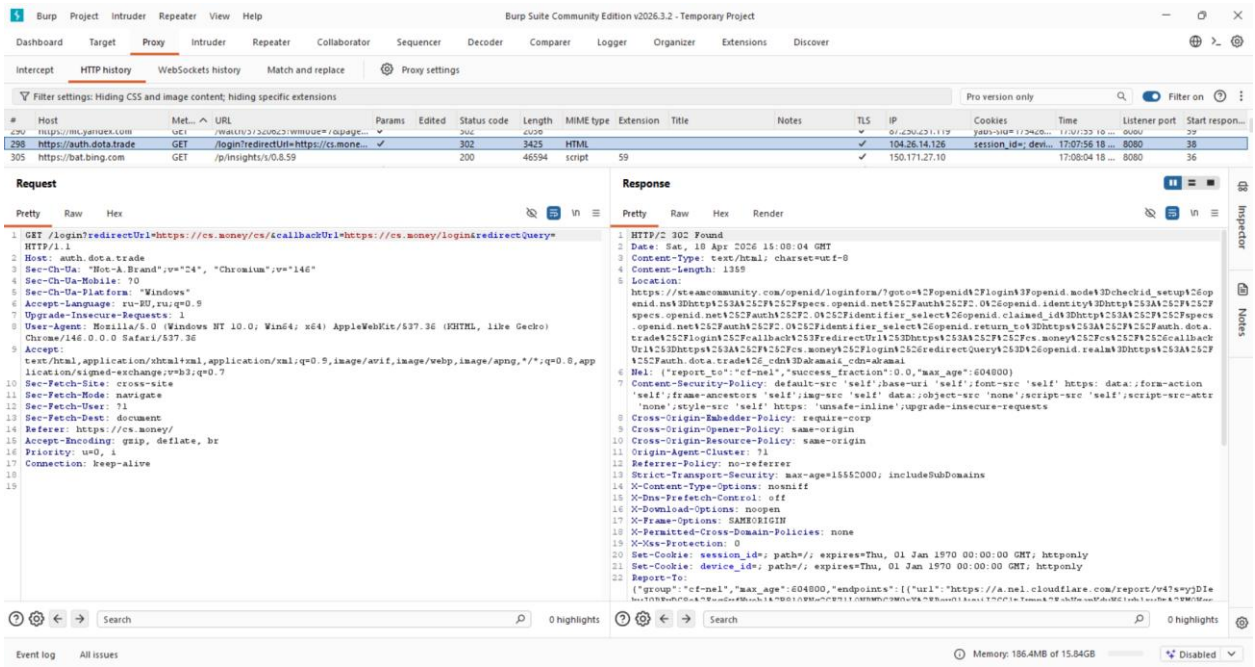

Figure 4: Intercepted authentication request to the intermediate service (auth.dota.trade) in Burp Suite

Figure 5: HTTP request details containing redirect parameters (redirectUrl, callbackUrl) during login initiation

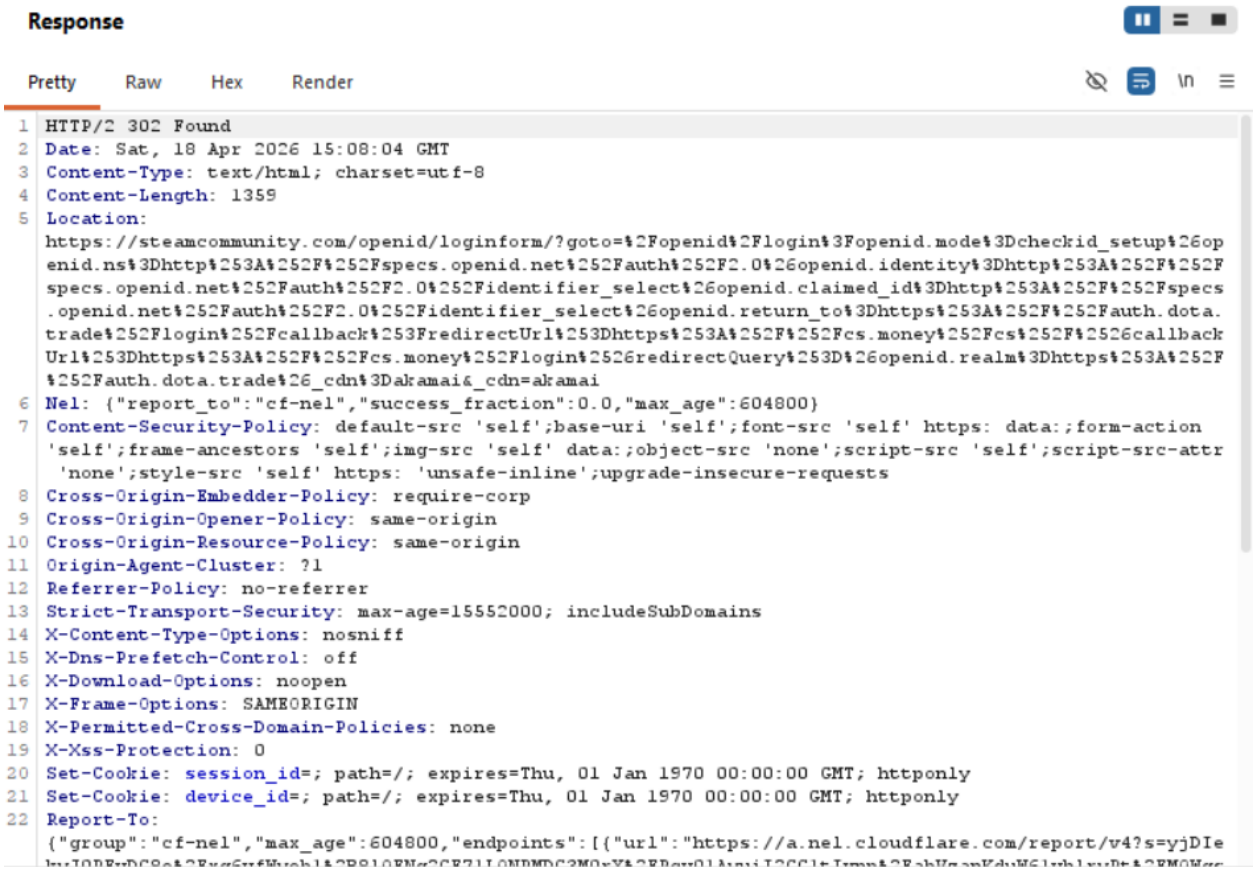

Figure 6: HTTP 302 response redirecting the user to Steam OpenID authentication endpoint

Identified Technologies (via reconnaissance)

During the reconnaissance phase, several technologies used by the cs.money platform were identified using passive analysis tools such as Wappalyzer and Burp Suite. https://www.wappalyzer.com/lookup/cs.money/

Frontend: The frontend of the application appears to be built using modern JavaScript frameworks. Wappalyzer detected the presence of Next.js and React, indicating a dynamic single-page application (SPA) architecture. Additionally, Bootstrap was identified as a UI framework.

Backend: The backend technology stack includes PHP, as identified by Wappalyzer. The application is likely supported by a MySQL database.

Infrastructure: The application is hosted behind a content delivery and security layer, likely provided by Cloudflare, based on observed network traffic and IP ranges. Additionally, Amazon Web Services (AWS) was detected as part of the hosting infrastructure.

Authentication: User authentication is handled via an external identity provider (Steam), using an OpenID-based authentication flow, as confirmed through intercepted HTTP traffic.

Security Mechanisms: The platform enforces HTTPS communication and utilizes HSTS (HTTP Strict Transport Security). Additionally, Cloudflare Turnstile was identified as a bot protection mechanism.

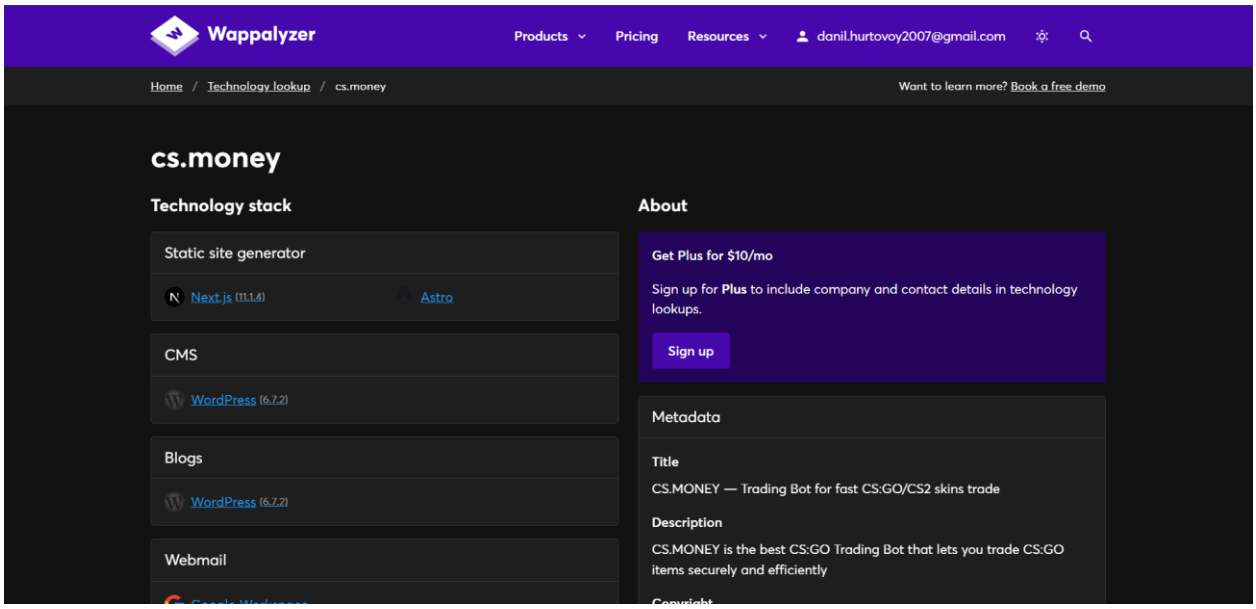

Figure 7: Identified frontend and CMS technologies (Next.js, WordPress) detected via Wappalyzer.

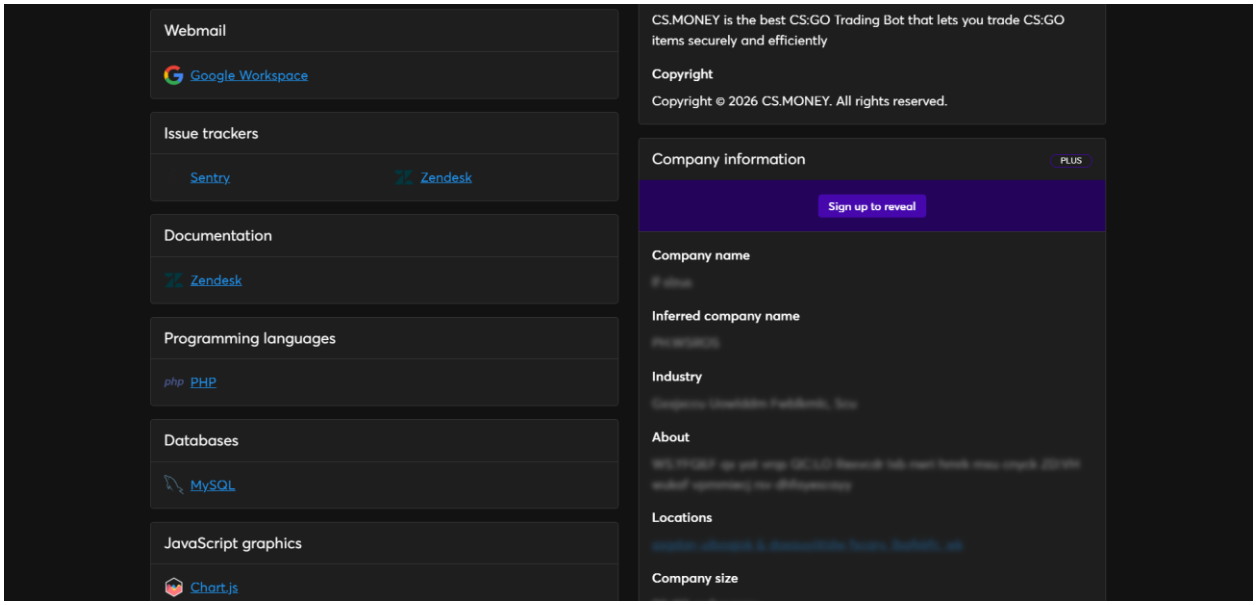

Figure 8: Identified backend-related technologies and infrastructure components (PHP, MySQL).

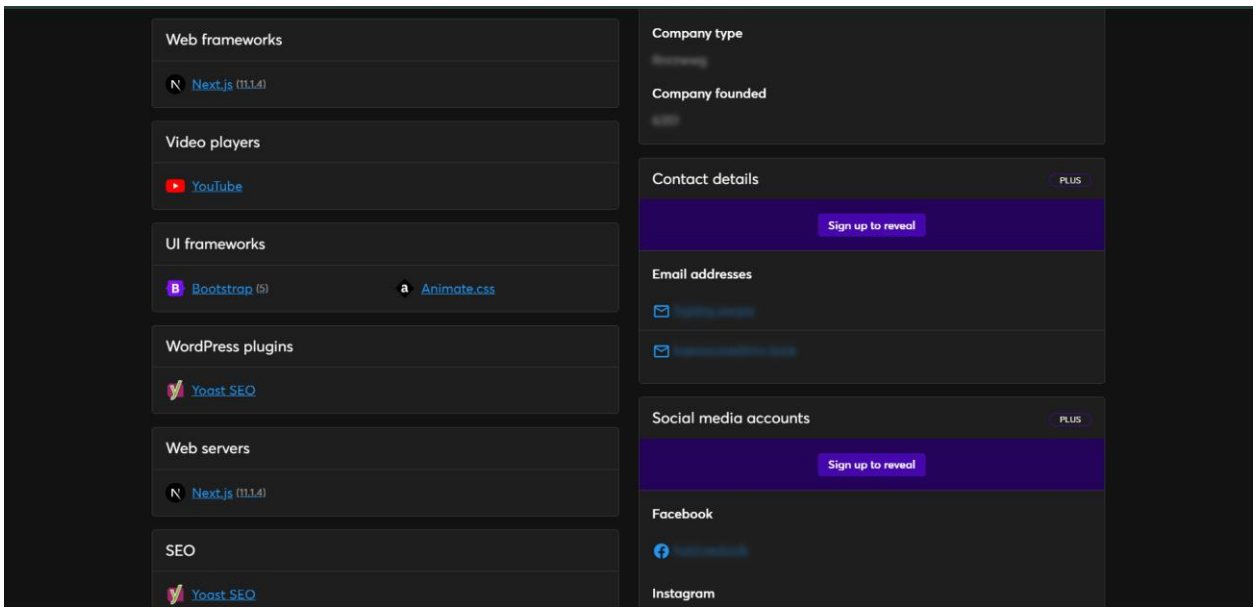

Figure 9: Identified UI frameworks and client-side technologies (Bootstrap, Animate.css, React).

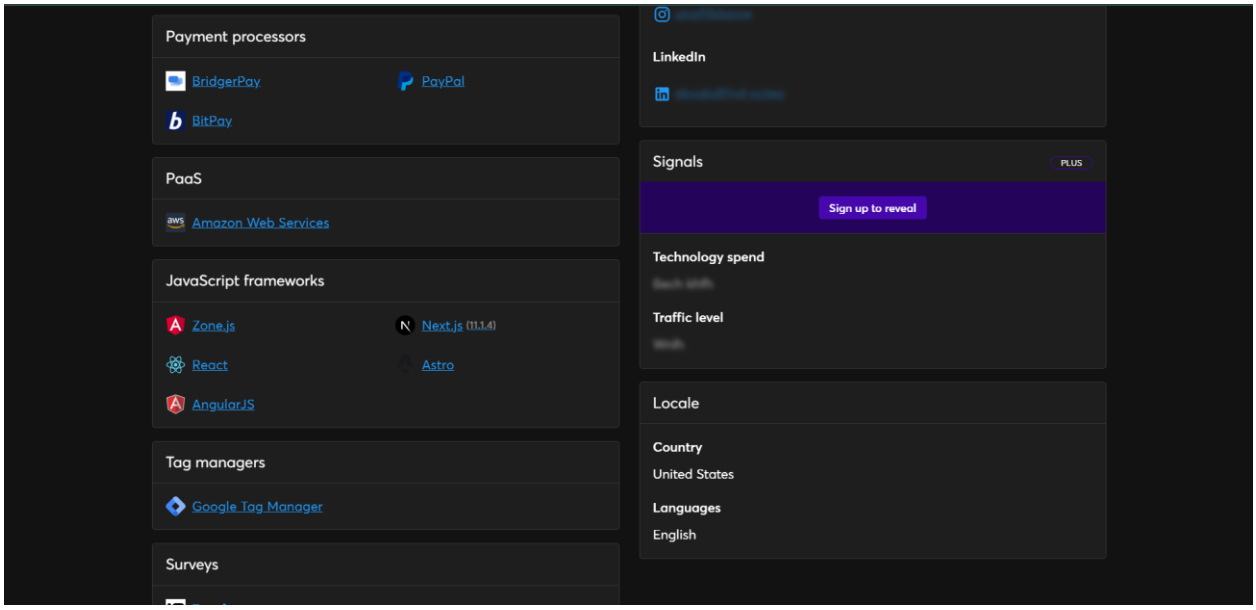

Figure 10: Identified third-party services and integrations (payment processors, analytics, tracking tools).

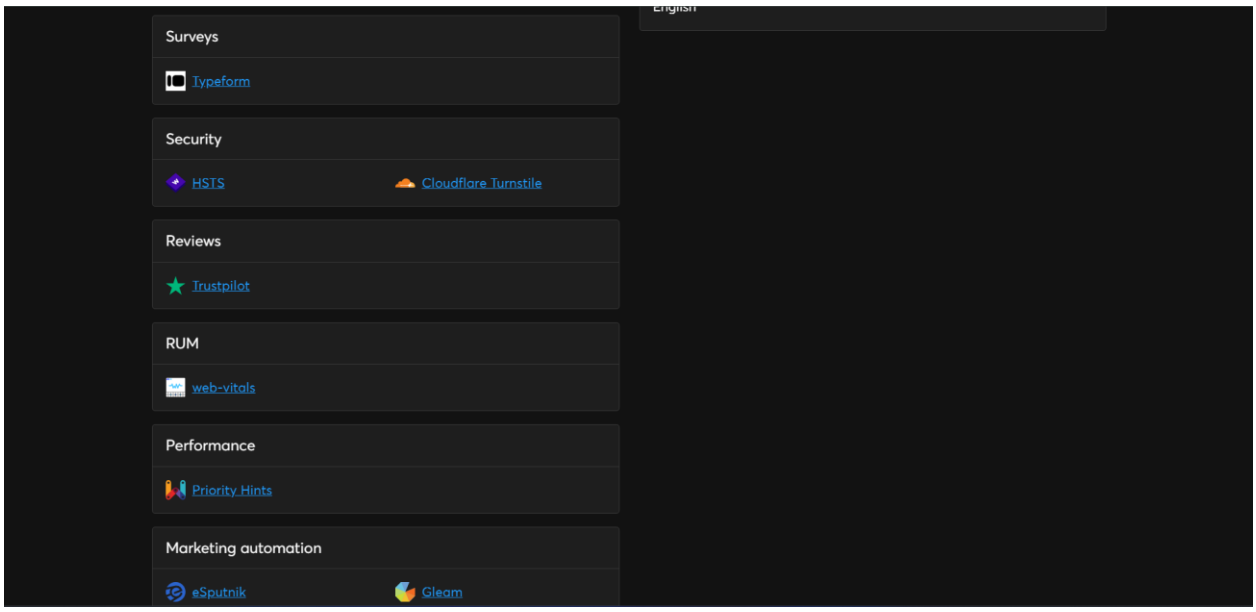

Figure 11: Identified security-related mechanisms (HSTS, Cloudflare Turnstile).

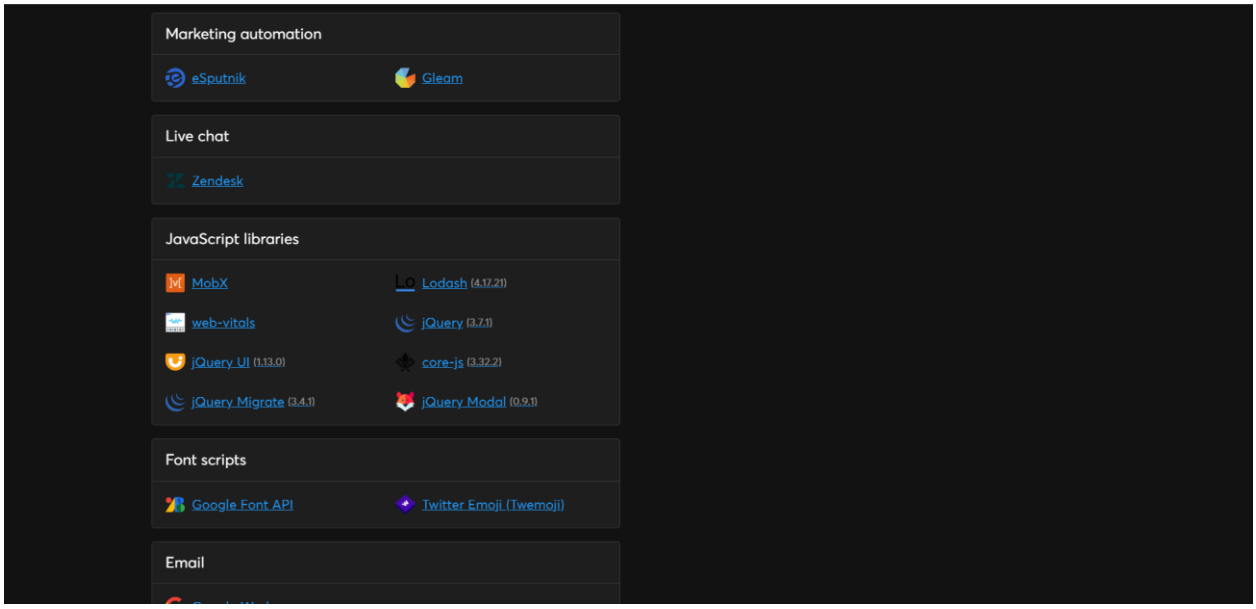

Figure 12: Identified JavaScript libraries and client-side dependencies (jQuery, Lodash, MobX, core-js).

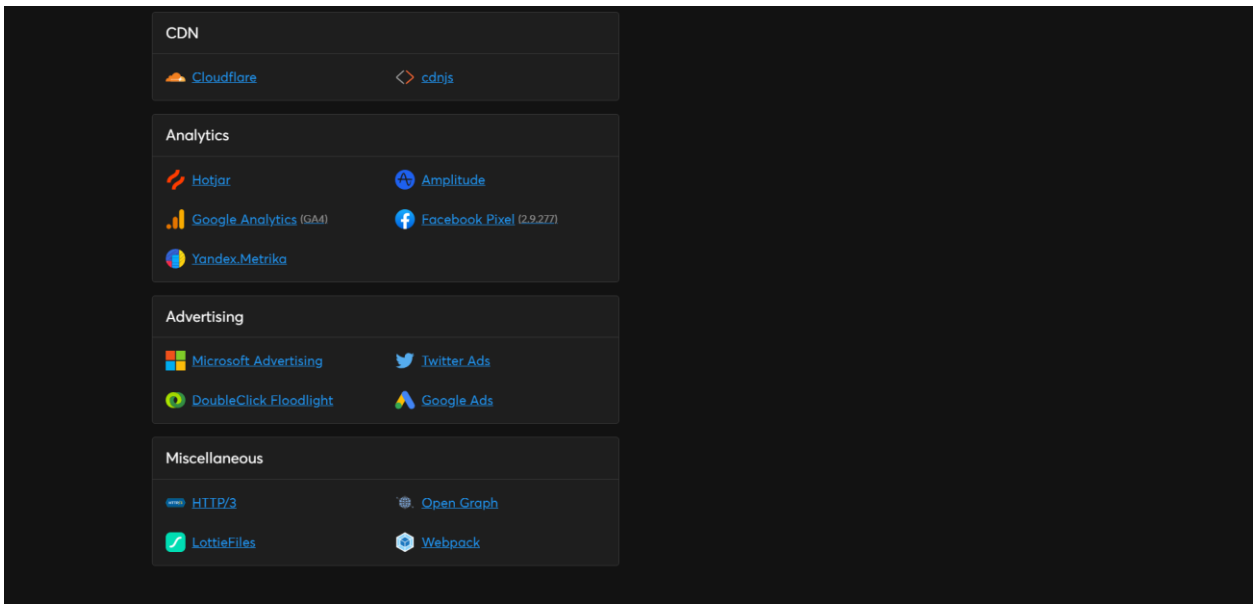

Figure 13: Identified CDN and delivery infrastructure (Cloudflare, cdnjs).

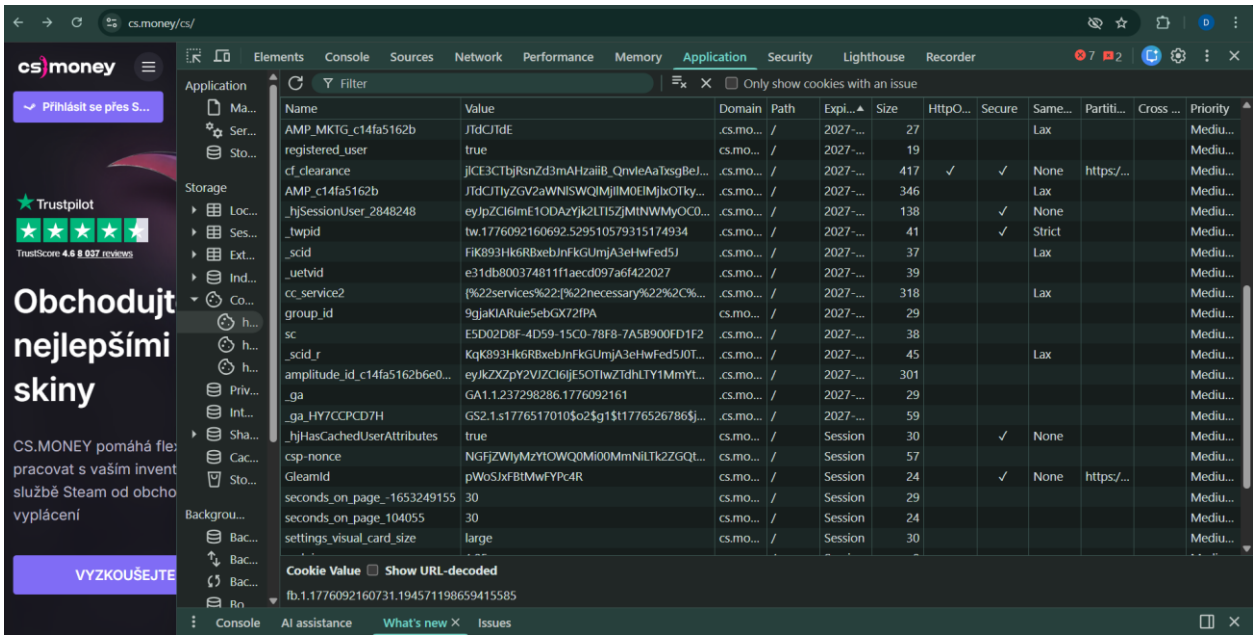

Figure 14: Cookies identified in the application, including session and tracking cookies with security attributes (HttpOnly, Secure, SameSite).

Assets Identified

During the reconnaissance phase, several key assets were identified based on application behavior and observed functionality:

- *User Accounts:* Registered users authenticated via Steam OpenID. Accounts are associated with user-specific data and actions within the platform.
- *User Inventory:* The platform interacts with users' Steam inventories, allowing items (skins) to be viewed, traded, and transferred.
- *Financial Assets:* Users interact with monetary values, including balances, pricing of items, and trade offers involving real-world value.
- *Trade Operations:* Core business logic revolves around item exchanges between users and the platform, representing a critical asset.
- *Session Data:* Session-related data maintained via cookies (e.g., session identifiers, tracking tokens), used for authentication and state management.
- *API Data:* Data exchanged between client and backend via API endpoints (e.g., user data, inventory data, trade information).
- *Third-party Authentication Data:* Authentication is handled via external Steam OpenID provider, including redirection tokens and authentication assertions.
- *Client-side Stored Data:* Data stored in browser (cookies, local/session storage), including tracking identifiers and session-related information.

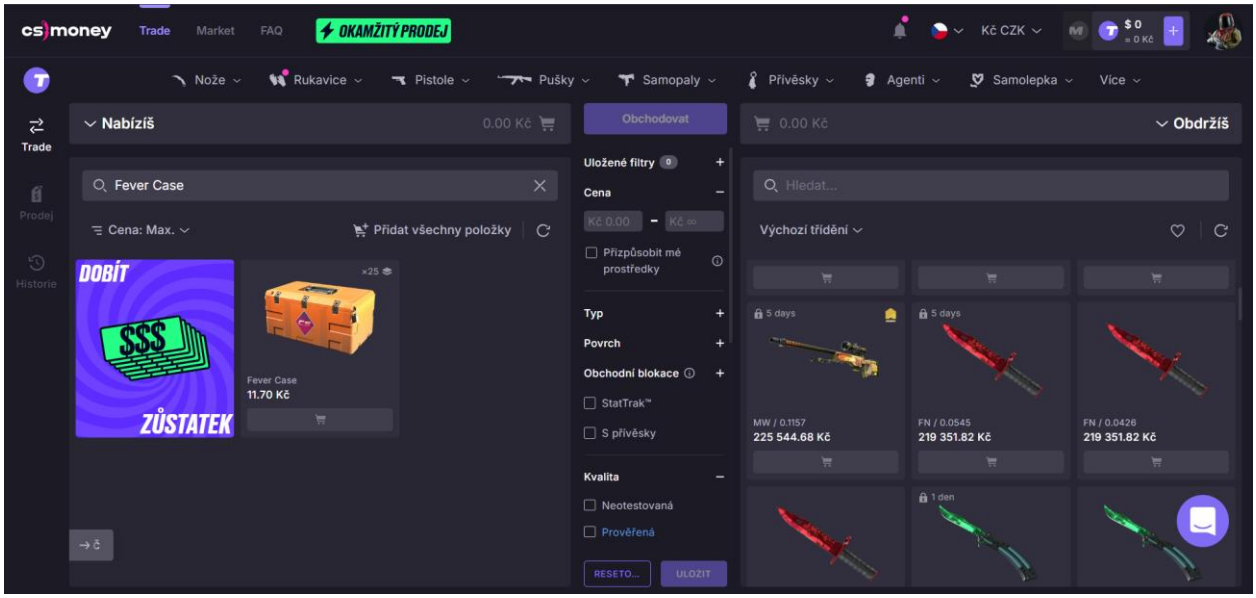

Figure 15: User inventory interface displaying tradable items (skins).

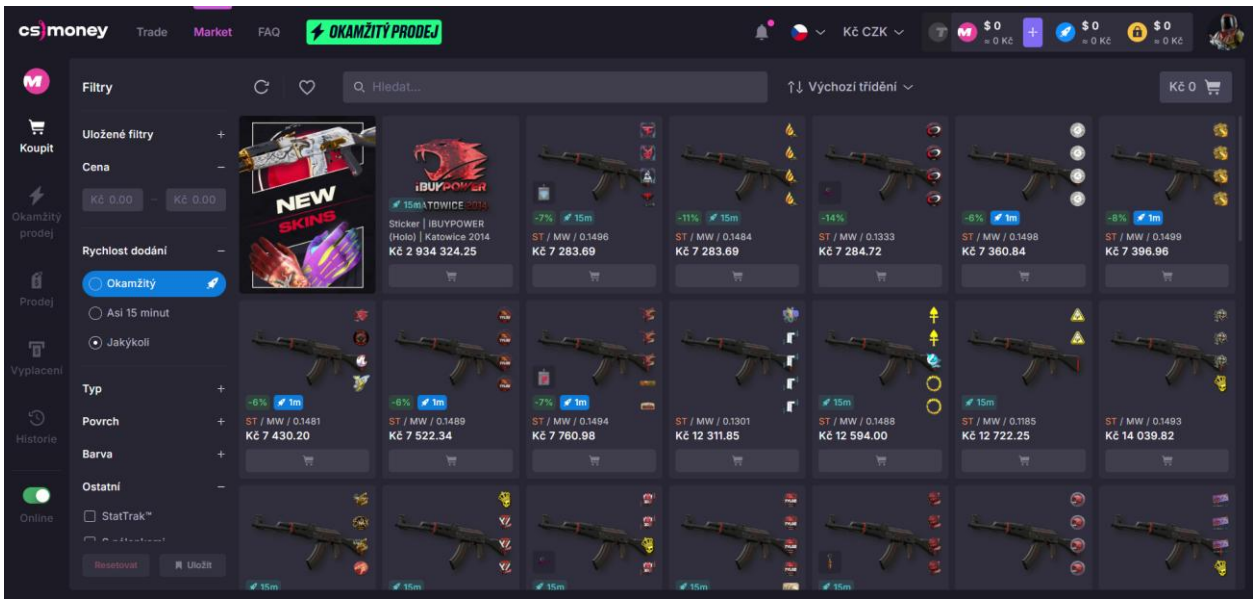

Figure 16: Trade interface showing item prices and exchange functionality.

External Integrations

During the reconnaissance phase, several third-party services and external integrations were identified:

- *Steam OpenID:* User authentication is handled via Steam using an OpenID-based authentication flow. This introduces dependency on an external identity provider.
- *Cloudflare:* The application is protected and delivered through Cloudflare, which provides CDN, DDoS protection, and bot mitigation (including Cloudflare Turnstile).
- *Payment Processors:* Third-party payment services such as PayPal, BitPay, and BridgerPay are used to handle financial transactions.
- *Analytics and Tracking Services:* Multiple analytics tools were identified, including Google Analytics, Facebook Pixel, Amplitude, and Hotjar, used for user behavior tracking.
- *Content Delivery Networks (CDN):* Static assets and scripts are delivered via CDN services such as Cloudflare and cdnjs.
- *Third-party JavaScript Libraries:* External libraries (e.g., jQuery, Lodash, core-js) are loaded from third-party sources, potentially introducing supply chain risks. These integrations expand the attack surface and introduce additional trust dependencies that must be considered during security assessment.

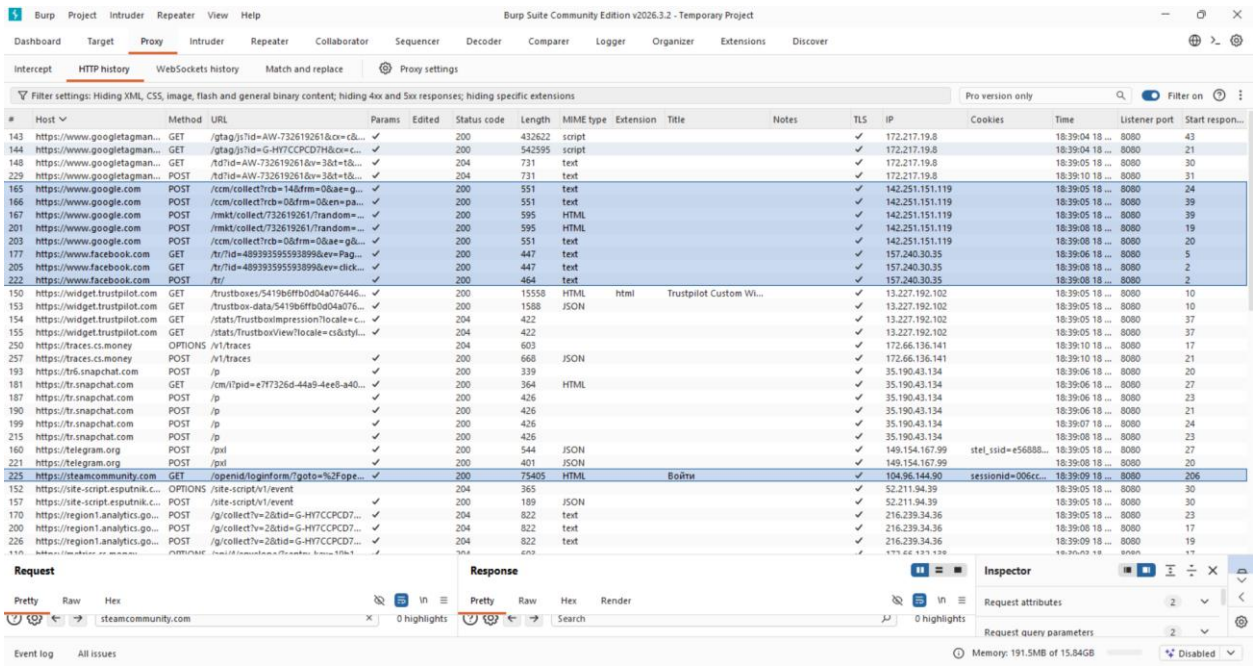

Figure 17: Third-party analytics and tracking services observed in HTTP traffic, including Google Analytics, Google Tag Manager, and Facebook Pixel.

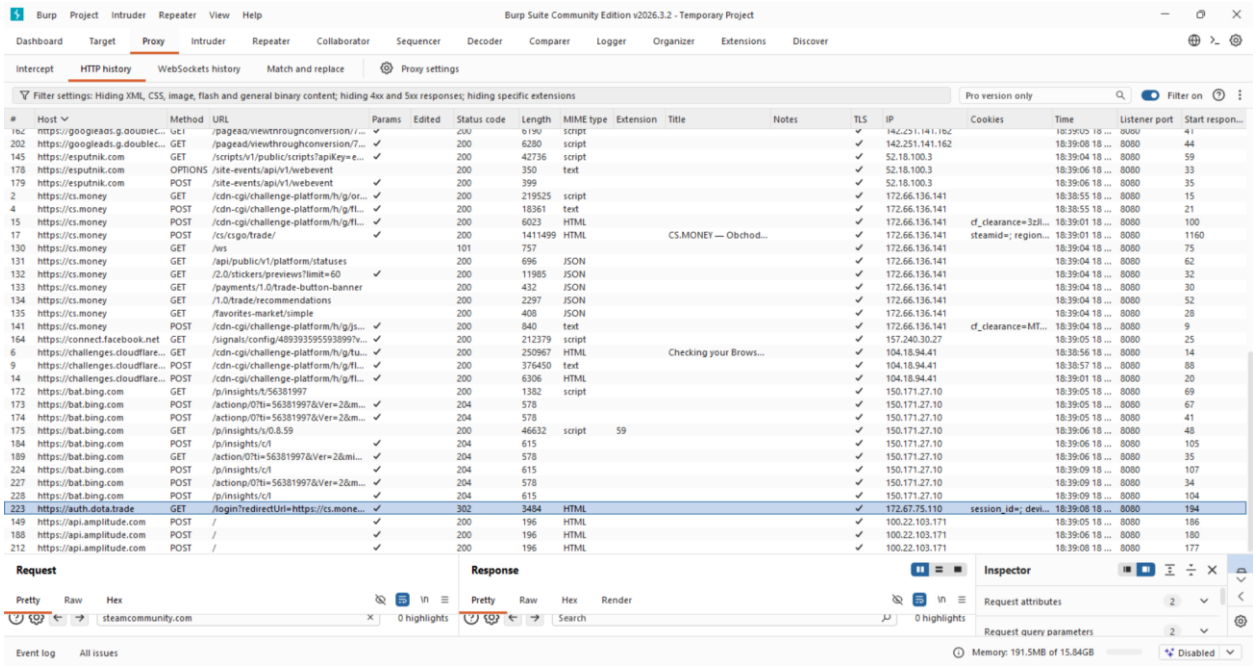

Figure 18: Intermediate authentication service observed, acting as a proxy between the application and external identity provider.

Hosting & Infrastructure Observations

Based on passive reconnaissance and traffic analysis, the following observations were made regarding the hosting and infrastructure of the cs.money application:

- The application is publicly accessible over HTTPS (TCP port 443), as confirmed by TLS traffic analysis in Wireshark.
- TLS certificates are issued by Let's Encrypt (E8), indicating automated certificate management.
- Traffic is routed through Cloudflare infrastructure, suggesting the use of CDN, DDoS protection, and Web Application Firewall (WAF) services.
- Multiple requests to Cloudflare challenge endpoints (e.g., /cdn-cgi/) were observed, indicating active anti-bot protection mechanisms.
- The application appears to use a distributed infrastructure with multiple IP addresses, consistent with CDN-backed delivery.
- External services such as Google Analytics, Facebook Pixel, and Steam OpenID are integrated, indicating reliance on third-party infrastructure components.

These observations suggest that the application is deployed behind a modern cloud-based infrastructure with layered security controls and external service integrations.

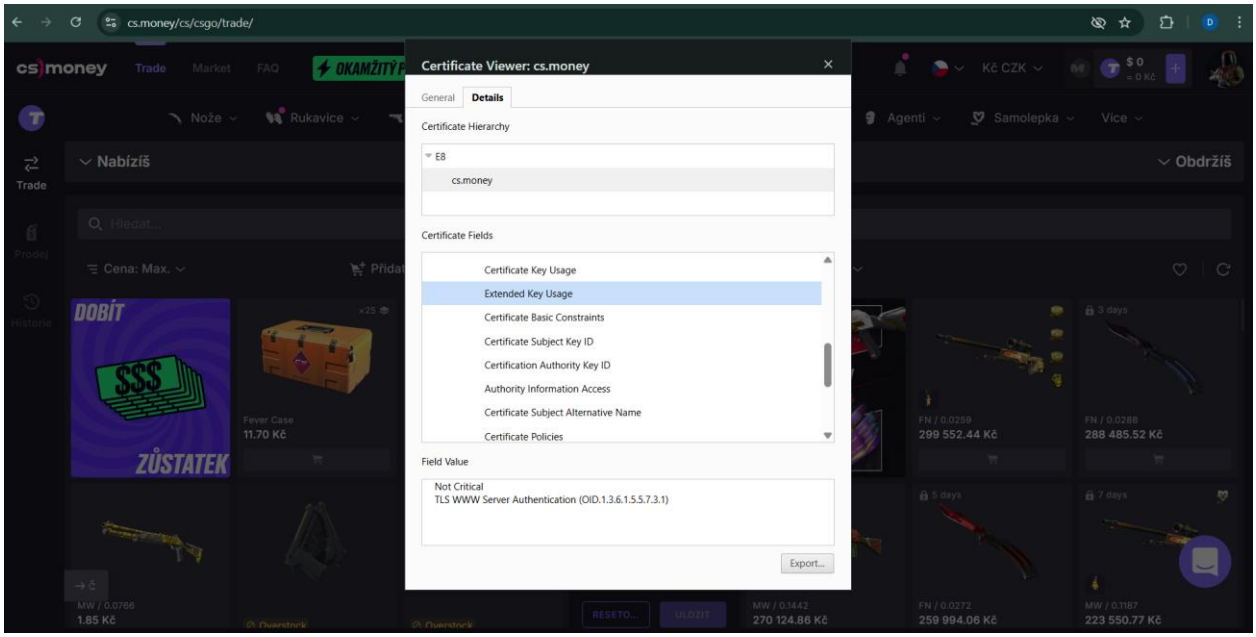

Figure 19: TLS certificate details for cs.money issued by Let's Encrypt.

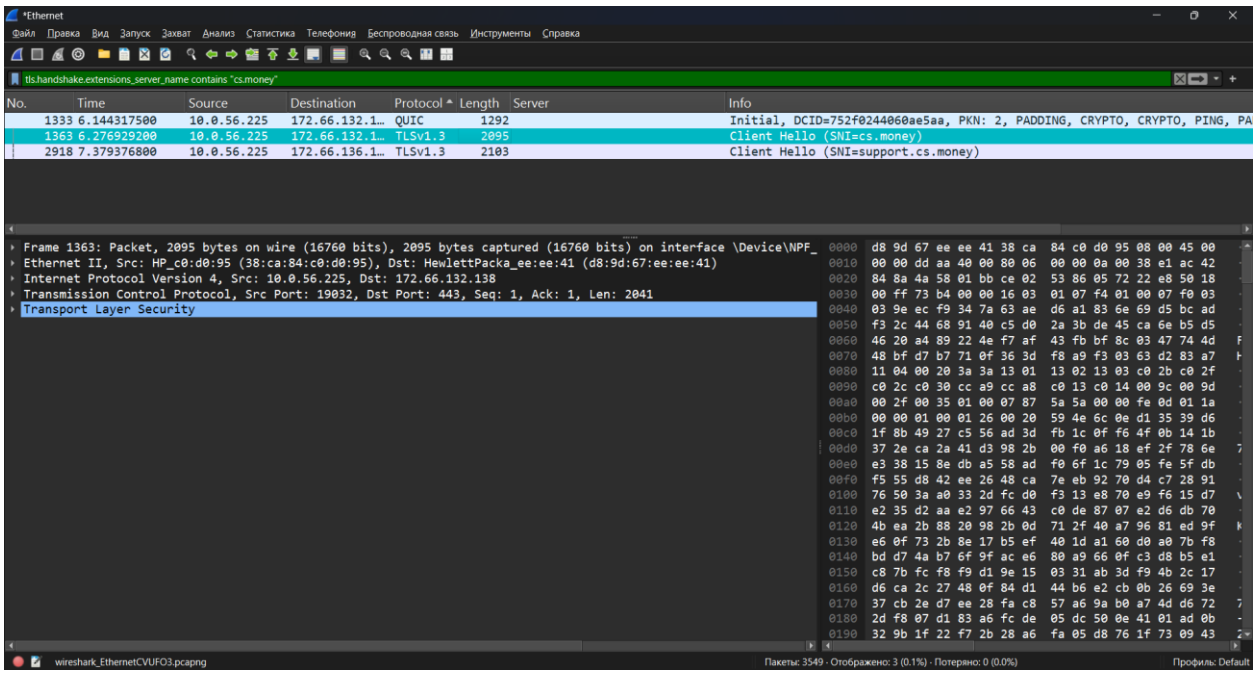

Figure 20: TLS handshake showing secure communication with cs.money over HTTPS (port 443), including Server Name Indication (SNI).

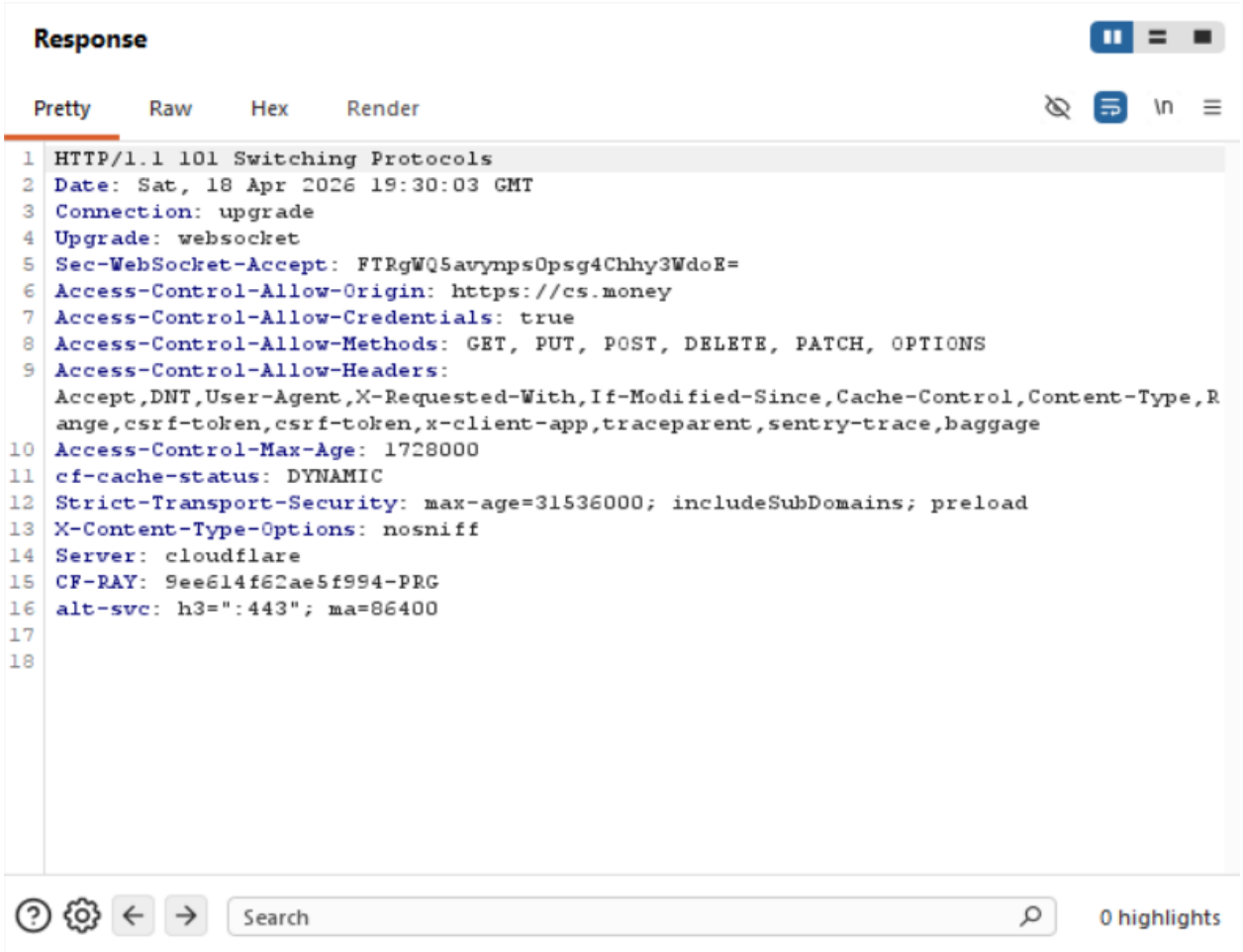

Figure 21: HTTP response headers indicating Cloudflare infrastructure, including caching, request tracing (cf-ray), and support for HTTP/3.

Entry Points

Based on traffic analysis and application interaction, the following entry points into the cs.money platform were identified:

- Public Web Interface: The main application is accessible via https://cs.money, providing user interaction through a browser-based interface.
- Authentication Endpoint: Login functionality is handled via redirection to an external OpenID provider (Steam), observed through requests to auth.dota.trade and steamcommunity.com.
- REST/JSON API Endpoints: The application communicates with backend services via JSON-based API calls (e.g., /api/, /payments/, /trade/ endpoints observed in HTTP traffic).
- WebSocket Communication: The presence of HTTP 101 Switching Protocols responses confirms the use of WebSocket connections for real-time communication.
- Static Asset Delivery: JavaScript, fonts, and other frontend resources are served via CDN infrastructure (Cloudflare and third-party services).
- Third-party Integration Endpoints: External endpoints for analytics and tracking (Google Analytics, Facebook Pixel) are actively used and represent additional interaction surfaces.

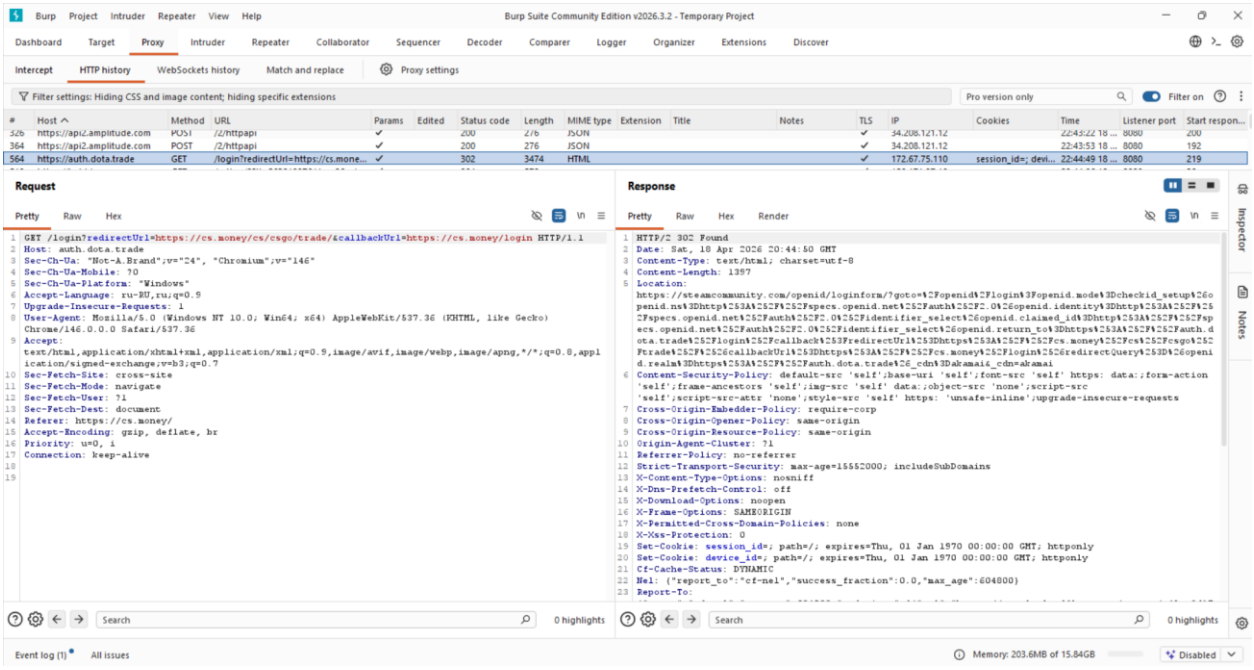

Figure 22: Authentication request redirected to external Steam OpenID provider.

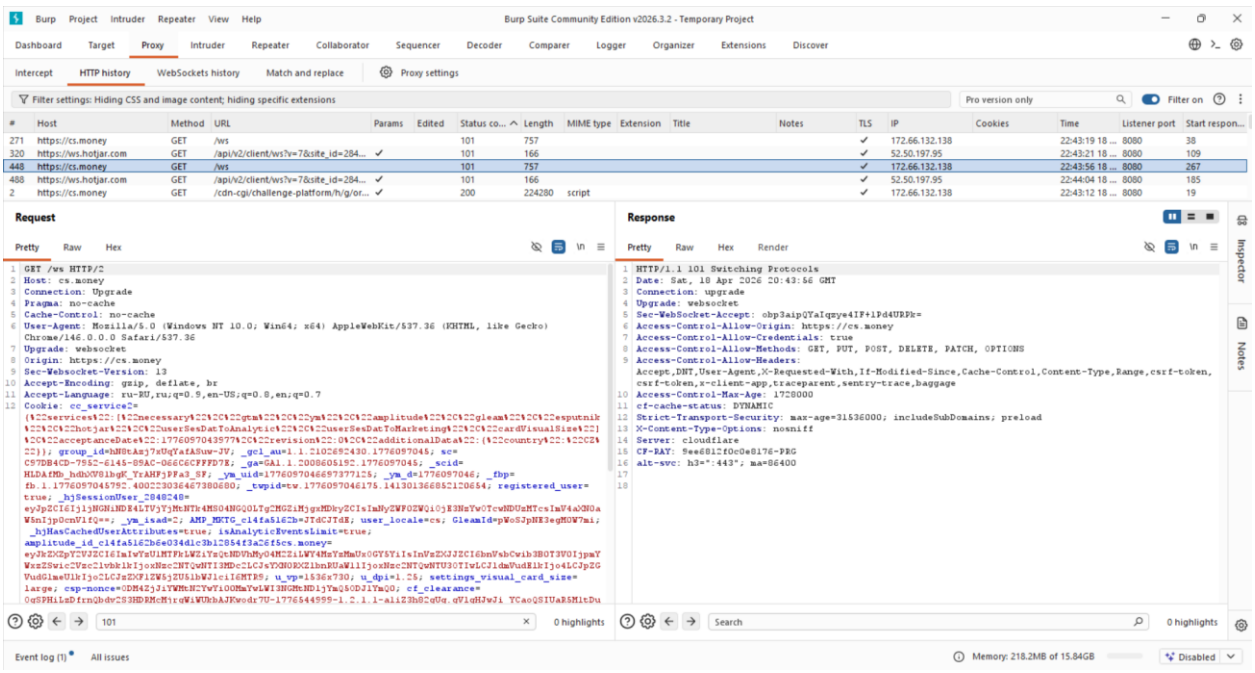

Figure 23: WebSocket upgrade observed on cs.money endpoint (/ws), confirming use of real-time communication via WebSockets.

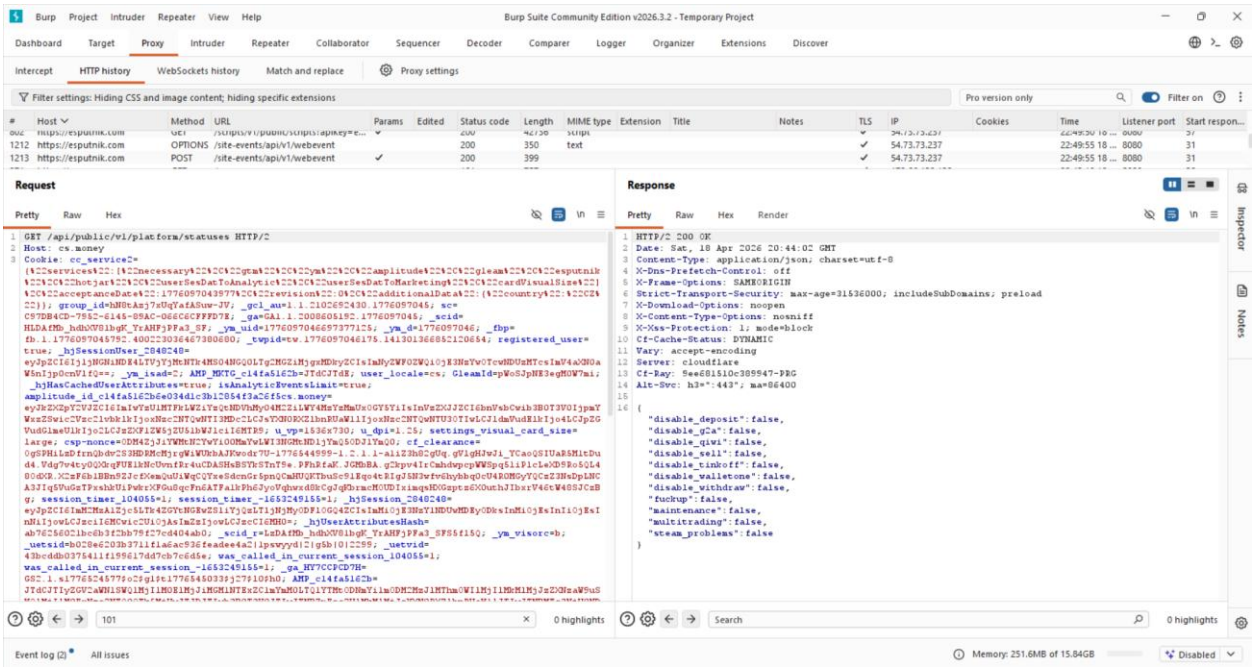

Figure 24: Example of a publicly accessible REST API endpoint (/api/public/v1/platform/statuses) observed via Burp Suite, returning structured JSON data and demonstrating backend functionality exposure.

High-Level Attack Surface

Based on reconnaissance activities, several high-level attack surfaces were identified within the cs.money application:

- Public REST/JSON API endpoints
  Multiple API endpoints (e.g., /api/public/v1/platform/statuses) were identified, returning structured JSON data. These endpoints may be vulnerable to:
  - parameter tampering
  - IDOR (Insecure Direct Object Reference)
  - improper access control
- Authentication flow (OpenID / OAuth via Steam)
  The authentication mechanism relies on external identity providers (Steam OpenID). Potential attack vectors include:
  - authentication bypass attempts
  - manipulation of redirect parameters
  - replay or misuse of authentication tokens
- WebSocket communication
  WebSocket connections (e.g., /ws endpoint) were observed, enabling real-time communication. These may introduce risks such as:
  - insufficient message validation
  - unauthorized access to real-time data streams
- Client-side storage and session handling
  The application utilizes cookies for session management and tracking. Observed risks include:
  - session hijacking
  - improper cookie configuration
  - exposure of sensitive data in client-side storage
- Third-party integrations
  The application heavily relies on external services (Google Analytics, Facebook, Hotjar, Cloudflare, etc.), which increases the attack surface through:
  - supply chain risks
  - malicious script injection via third-party resources

- CDN and infrastructure layer (Cloudflare)
  The presence of Cloudflare indicates the use of CDN and security filtering. However:
    - origin server exposure
    - misconfigured security rules

  may still introduce vulnerabilities
- Input vectors in URL parameters and API requests
  Parameters such as redirectUrl and callbackUrl were observed in authentication requests, which may be vulnerable to:
    - open redirect attacks
    - parameter manipulation

These identified attack surfaces define the primary focus areas for subsequent penetration testing activities.

Attack Surface Indicators

The following indicators of potential security weaknesses were identified during reconnaissance:

- Presence of redirectUrl and callbackUrl parameters in authentication flow
- Public API endpoints returning structured JSON data
- Extensive use of cookies for session and tracking purposes
- WebSocket communication channels observed (101 Switching Protocols)
- Exposure of platform state information via API responses
- Requests to multiple third-party services (analytics, tracking, CDN)
- Use of Cloudflare as CDN and reverse proxy

These indicators highlight areas requiring further security testing.

Conclusion

The reconnaissance phase revealed the main components of the cs.money platform, including authentication via Steam OpenID, publicly accessible API endpoints, and real-time communication through WebSockets.

The application relies on a distributed infrastructure with Cloudflare protection and integrates multiple third-party services.

Identified entry points and observed behaviors highlight several areas for further testing, particularly in API interactions, authentication flow, and session handling.

Structure of the final report

This repository contains a sanitized methodology and report structure for a black-box web application security testing project. It does not include real target details, private endpoints, exploit payloads, or confirmed vulnerability findings, in order to follow responsible disclosure principles and bug bounty program rules. The intended final report is structured to document confirmed findings, if identified during testing, by vulnerability type, severity, evidence, business impact, and recommended mitigation. It also includes sections for methodology, scope, tested areas, risk assessment, and an overall security evaluation of the application.